

Processes and Threads

Resource Ownership

A process has a virtual address space to store its process image, and the OS protects resources to prevent unwanted interference between processes.

Scheduling/Execution

A process has an execution path that may mix with others. It has an execution state (like Running or Ready) and a priority for scheduling, and it's managed by the OS for execution.

- The **unit of dispatching** is referred to as a **thread** or **lightweight process**
- The **unit of resource ownership** is referred to as a **process** or **task**
- **Multithreading** - The ability of an OS to support multiple, concurrent paths of execution within a single process

Single Threaded Approaches

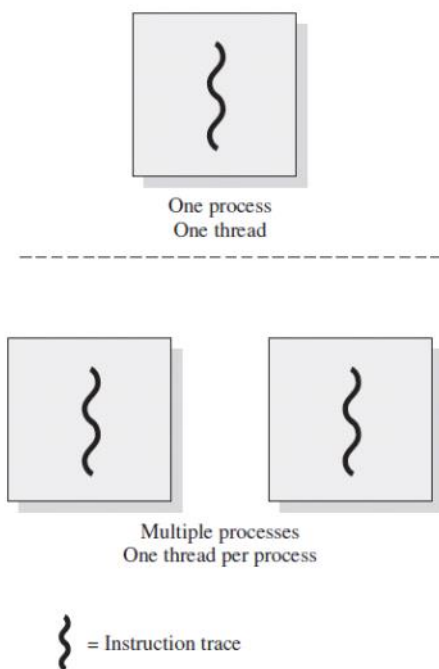
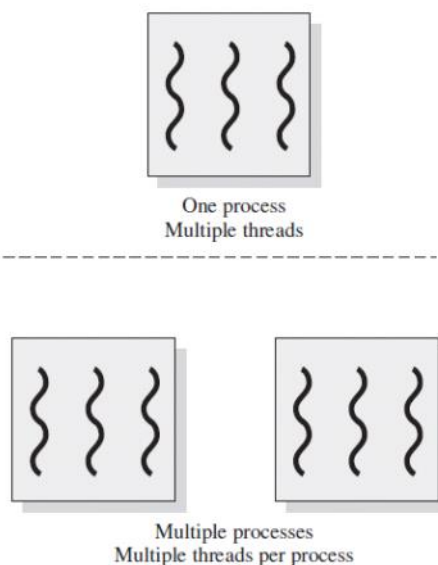


Figure 4.1 Threads and Processes

Multithreaded Approaches



- A single thread of execution per process, in which the concept of a thread is not recognized, is referred to as a single-threaded approach.
- MS-DOS is an example. (single T.)
- A Java run-time environment is an example of a system of one process with multiple threads

Process

In a multithreaded environment, a process is defined as the **unit of resource allocation and a unit of protection**. The following are associated with processes:

- A virtual address space that holds the process image
- Protected access to processors, other processes (for interprocess communication), files, and I/O resources (devices and channels)

One or More Threads in a Process

Each thread has:

- An execution state (Running, Ready, etc.)
- A saved thread context when not running
- An execution stack
- Some per-thread static storage for local variables
- Access to the memory and resources of its processes, shared with all other threads in that process

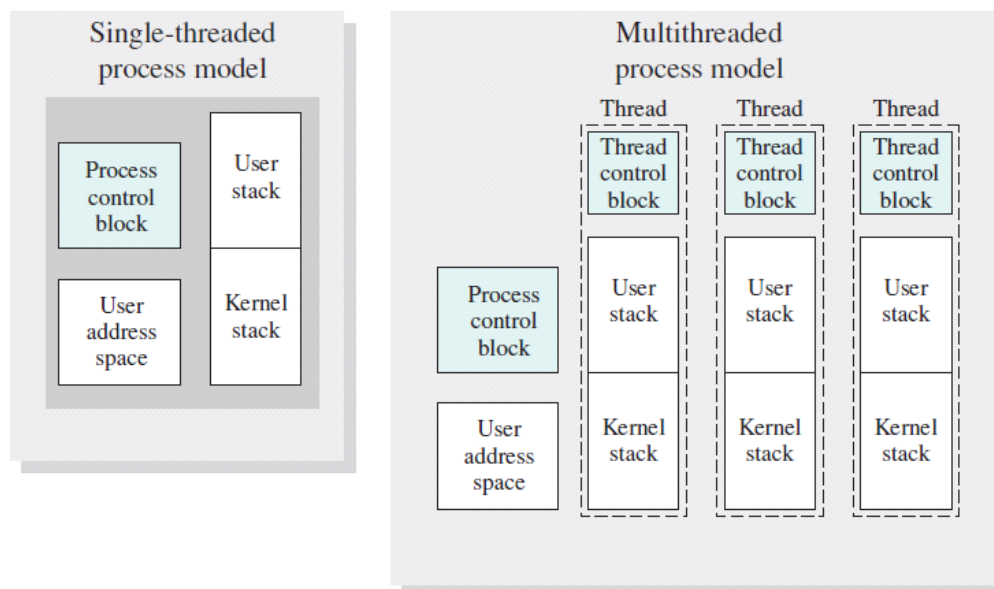
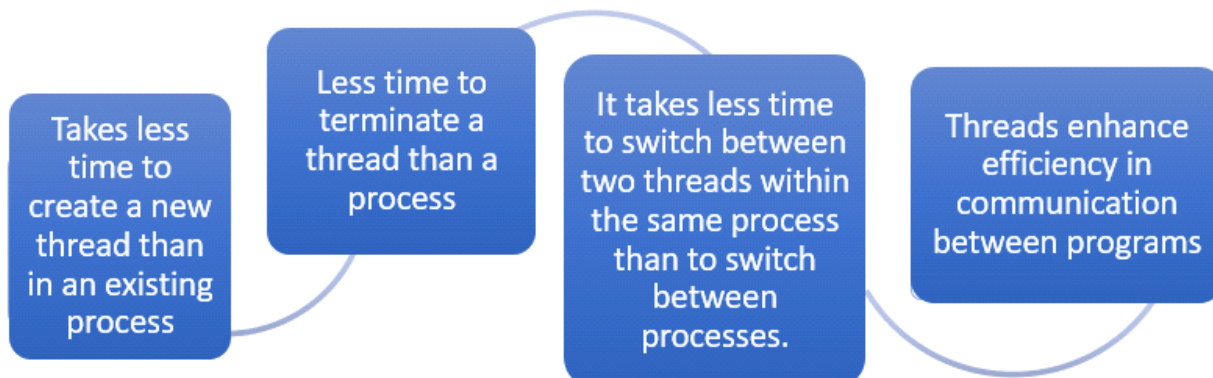


Figure 4.2 Single-Threaded and Multithreaded Process Models

! Key Benefits of Threads !



Using threads is faster than using processes because threads are **smaller units** within the same process and require **fewer CPU resources**. This allows for **faster** and more **efficient** parallel processing.

Thread Use in a Single-User System

- **Foreground and Background Work:** Using threads lets the app handle **user actions and commands at the same time**, making it seem faster.
- **Asynchronous Processing:** Threads allow regular tasks, like auto-saving, to run separately without making the main program complex.
- **Speed of Execution:** Multithreaded programs can do **I/O and calculations at the same time**, especially helpful on systems with multiple processors.
- **Modular Program Structure:** Threads make it **easier to organize programs** that manage different tasks, inputs, and outputs.

Threads

- In a thread-supporting OS, scheduling and managing is handled per thread, with most execution details stored at the thread level.
- When a process is paused(suspend) or stopped(terminated), all its threads are also paused or stopped.

Thred Execution States

The key states for a thread are:

- Running
- Ready
- Blocked

Thread operations associated with a change in thread state are:

- ☐ Spawn
- ☐ Block
- ☐ Unblock
- ☐ Finish

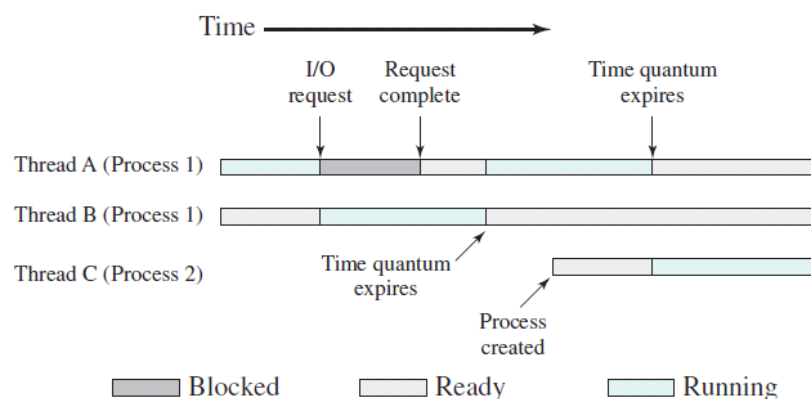


Figure 4.4 Multithreading Example on a Uniprocessor

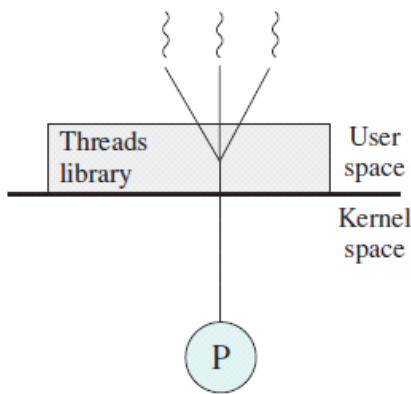
Multiprogramming on a uniprocessor allows multiple threads across different processes to run in an interleaved fashion. Here, three threads from two processes take turns using the processor. Execution switches to another thread when the current one is blocked or its time slice runs out.

Thread Synchronization

- It is necessary to synchronize the activities of the various threads
 - All threads of a process share the same address space and other resources (like open files).
 - Any change of a resource by one thread affects the other threads in the same process.

There are two types of Threads ; User Level Thread(ULT) , Kernel Level Thread(KLT).

User-Level Threads (ULTs)



(a) Pure user-level



- All thread management is done by the application
- The kernel is not aware of the existence of threads
- A threads library, included in the application, provides functions to manage threads, like creating (spawn function), destroying, passing messages, scheduling, and saving/restoring contexts.
- The threads library creates a data structure for the new thread and schedules it by switching control to a thread in the Ready state.

Advantages of ULTs

- Thread switching does not require kernel mode privileges.
- Scheduling can be application specific.
- ULTs can run on any OS.

Disadvantages of ULTs

When a User-Level Thread (ULT) makes a blocking system call, that thread gets blocked, which also causes all other threads within the same process to become blocked.

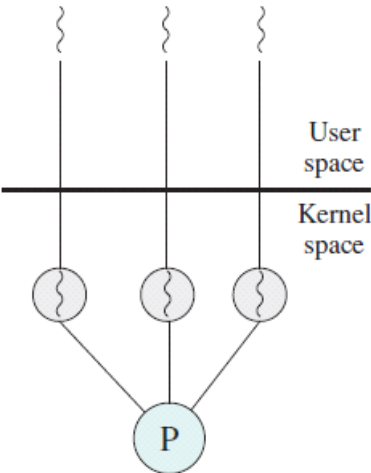
- In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing

Overcoming ULT Disadvantages

(Jacketing)

- Purpose is to convert a blocking system call into a non-blocking system call
- Writing an application as multiple processes rather than multiple threads

Kernel-Level Threads (KLTs) !



(b) Pure kernel-level



- Thread management is done by the kernel.
- There is no thread management code in the application level, simply (API) to the kernel thread facility.
- **Windows** is an example of this approach.

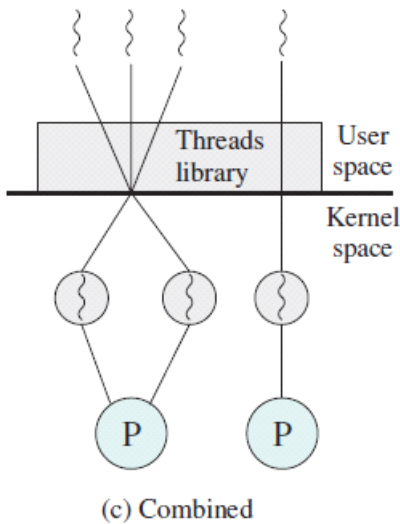
Advantages of KLTs !

- The kernel can simultaneously schedule multiple threads from the same process on multiple processors.
- If one thread in a process is blocked, the kernel can schedule another thread of the same process.
- Kernel routines themselves can be multithreaded.

Disadvantage of KLTs

- The transfer of control from one thread to another within the same process requires a mode switch to the kernel

Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840



Thread creation, scheduling, and synchronization are primarily handled in user space

- Solaris is a good example

{ User-level thread (wavy circle) Kernel-level thread (P) Process

Table 4.2 Relationship between Threads and Processes

Threads: Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	It combines attributes of M:1 and 1:M cases.	TRIX

Applications That Has Benefit

- Multithreaded native applications
 - Characterized by having a small number of highly threaded processes.
- Multiprocess applications
 - Characterized by the presence of many single-threaded processes.
- Java applications
 - All applications that use a Java 2 Platform, Enterprise Edition application server can immediately benefit from multicore technology.
- Multi-instance applications
 - Multiple instances of the application in parallel.

Windows Process and Thread Management

- An **application** consists of one or more processes
- Each **process** provides the resources needed to execute a program
- A **thread** is the entity within a process that can be scheduled for execution
- A **job object** allows groups of process to be managed as a unit
- A **thread pool** is a collection of worker threads that efficiently execute asynchronous callbacks on behalf of the application
- A **fiber** is a unit of execution that must be manually scheduled by the application
- **User-mode scheduling (UMS)** is a lightweight mechanism that applications can use to schedule their own threads

Management of Background Tasks and Application Lifecycles

- Starting with Windows 8 and continuing into Windows 10, developers manage the state of their applications.
- Previous Windows versions allowed users full control over the lifetime of processes.
- In the new **Metro interface**, Windows controls the application lifecycle.
 - A limited number of applications can run alongside the main app using SnapView.
 - Only one Store application can run at a time.
- **Live Tiles** create the illusion of constantly running applications.
 - However, they only receive push notifications and do not use system resources for dynamic content.

Metro Interface

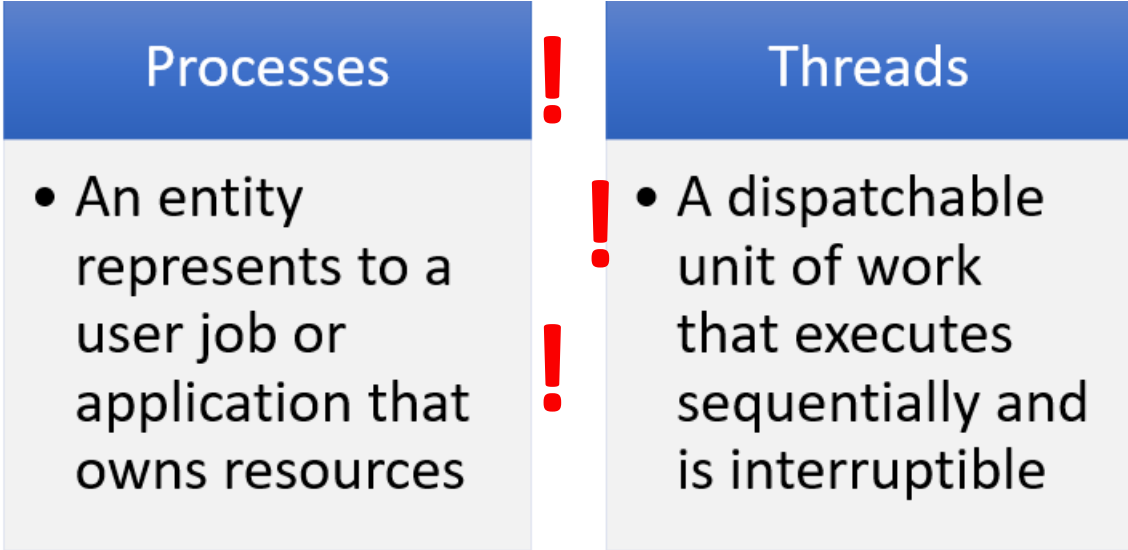
- The foreground application in the Metro interface has full access to processor, network, and disk resources.
- All other apps are suspended and cannot access these resources.
- When an app is suspended, it must trigger an event to save the user's information, which is the developer's responsibility.
- Windows can terminate background apps, so developers need to save the app's state when suspended.
- When the app returns to the foreground, another event retrieves the user state from memory.

Windows Process

Important characteristics of Windows processes are:

- Windows processes are implemented as objects
- A process can be created as a new process or a copy of an existing process
- An executable process may contain one or more threads
- Both process and thread objects have built-in synchronization capabilities

Windows makes use of two types of process-related objects:



Process ID
Security descriptor
Base priority
Default processor affinity
Quota limits
Execution time
I/O counters
VM operation counters
Exception/debugging ports
Exit status

Windows Process Object Attributes

Thread ID
Thread context
Dynamic priority
Base priority
Thread processor affinity
Thread execution time
Alert status
Suspension count
Impersonation token
Termination port
Thread exit status

Windows Thread Object Attributes

!

Multithreading

!

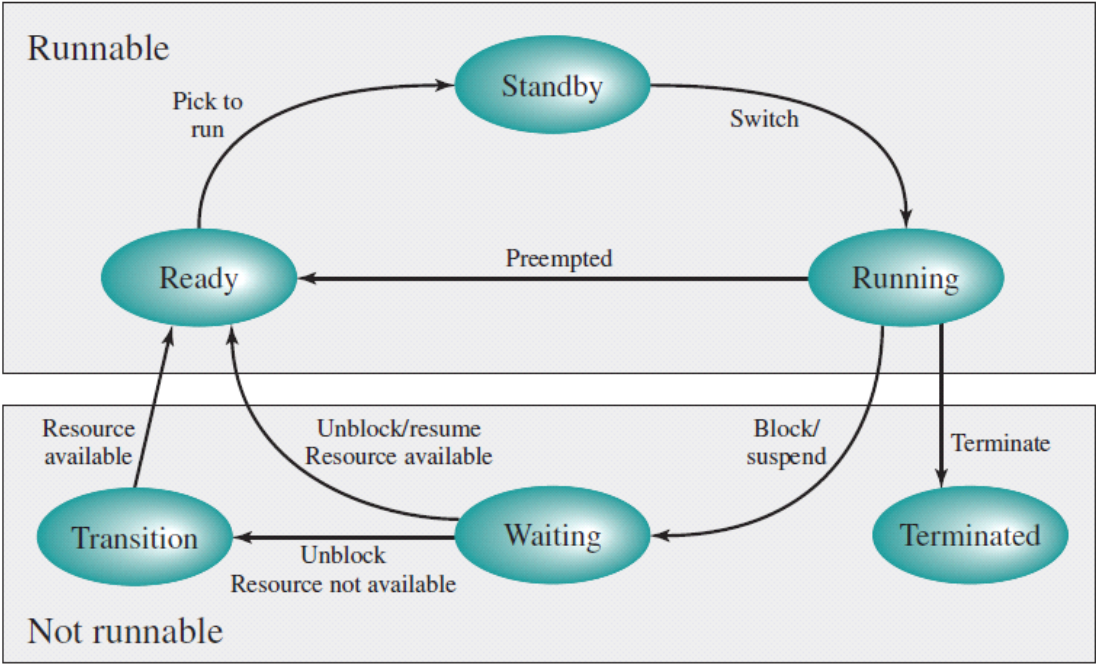
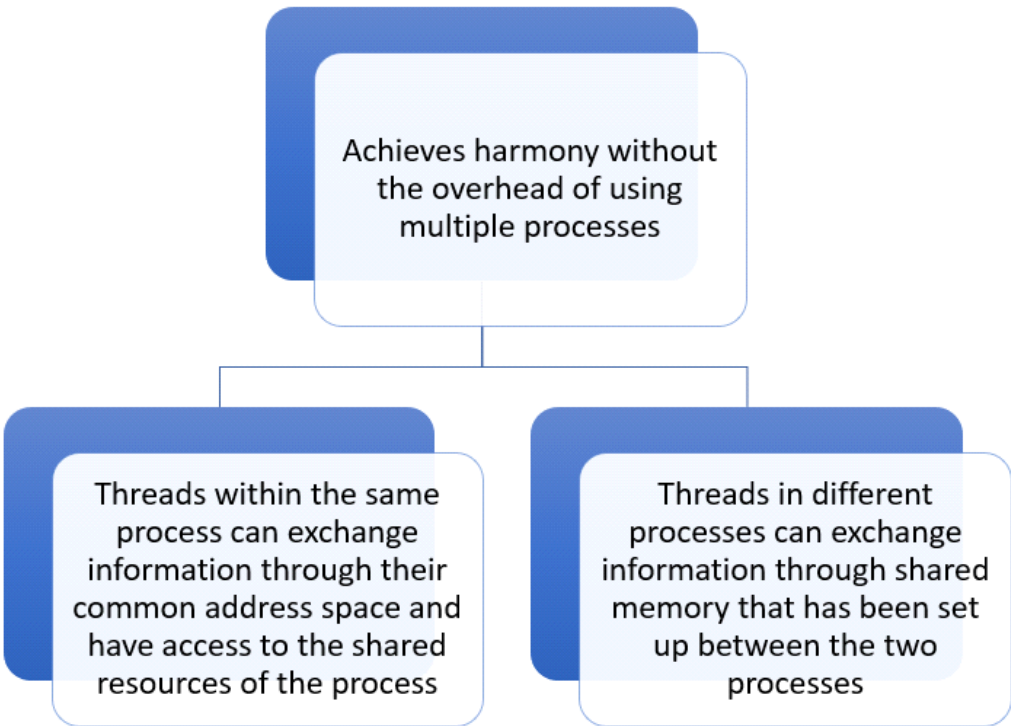


Figure 4.11 Windows Thread States